

Redis Client Capacity Exhaustion - Sanitized Incident Review

INC-REDIS-2026-07-A | Owner: Commerce Platform SRE
Internal training fixture - identifiers and traffic values sanitized
Reviewed: 2026-07-21 | Trace: INC-REDIS-2026-07-A / CR-REDIS-2026-071

1. Executive Summary and Impact

On 2026-07-06, order-service experienced elevated checkout failures after a release increased retry concurrency in the promotion lookup path. Redis client slots approached the configured maxclients ceiling, new connections waited or failed, and retries amplified the pressure.

The incident affected the order submission path for 18 minutes. Peak HTTP 5xx reached 12.8%, P95 latency rose from 420 ms to 4.6 s, and approximately 7.4% of checkout attempts required a user retry. No data loss was observed.

This review is retrospective evidence. A current healthy Redis INFO result only proves adapter connectivity; it does not prove that the historical incident is still active.

Scope	Observed impact
Service	order-service / promotion lookup
Window	2026-07-06 10:00-10:18 UTC
Customer effect	Checkout timeout, 5xx, delayed order confirmation
Peak signals	connected_clients=9940, maxclients=10000, blocked_clients=37
Data integrity	No confirmed loss or duplicate order writes

2. Incident Timeline

The timeline correlates deploy history, Prometheus metrics, Loki logs, and Redis INFO captured in the same incident window. Values outside this window were treated as supporting context rather than direct proof.

UTC	Event and evidence
09:10	order-api rc1 raised promotion lookup retry count
10:00	HighErrorRate and RedisConnectionWait alerts began firing
10:03	connected_clients crossed 9,900; blocked_clients reached 37
10:06	Loki showed Redis timeout and connection-pool wait messages
10:09	Incident commander froze unrelated production changes
10:12	Retry reduction plan approved; canary configuration prepared
10:15	Canary error rate and client growth began falling
10:18	5xx and P95 returned below recovery thresholds
10:28	rc2 completed after observation and approval

3. Evidence Register

The incident commander recorded each artifact with its observation window and diagnostic role. Historical tickets were supporting context only and were not treated as proof of the live incident.

Evidence	Observation	Supports	Limit
Prometheus	Clients 9,940/10,000; blocked=37	Capacity pressure	Does not identify owner
Loki	Pool acquire timeout after rc1	Application impact	Sampled error logs
Redis INFO	Rejected connections increased	Server-side refusal	Point-in-time snapshot
Deploy history	Retry concurrency changed at 09:10	Candidate trigger	Correlation alone
Canary	Client growth and 5xx declined	Causal reversal	10% traffic scope

4. Hypothesis Review and Root Cause

Hypothesis A - Redis CPU saturation: rejected. Redis CPU remained below 48% and command latency did not rise before client-slot pressure.

Hypothesis B - network packet loss: rejected. Node and load-balancer packet loss stayed within baseline, and failures correlated with connection creation.

Hypothesis C - a single slow command: not primary. Slowlog had no command whose timing explained the connection growth or the application pool waits.

Hypothesis D - client-capacity exhaustion caused by retry amplification: confirmed. The release increased retry concurrency, idle connections were retained longer than expected, `connected_clients` approached `maxclients`, and the rollback of retry behavior reversed all three signals.

Root cause: an application retry-policy change multiplied concurrent Redis connection demand while the pool retained idle connections. The combined load exhausted available client capacity and produced a positive feedback loop.

Diagnostic conclusion: the confirmed hypothesis is supported by temporal ordering, mechanism evidence, and causal reversal. The rejected hypotheses remain documented to prevent future responders from repeating the same checks.

5. Response, Approval, and Follow-up

Immediate response was evidence-first: preserve the incident window, identify connection owners, freeze unrelated changes, and prepare a reversible canary. The agent did not restart Redis or change maxclients automatically.

Approved change CR-REDIS-2026-071 authorized reducing retry count and idle-pool retention on a 10% canary. The approver was the incident commander with Redis service owner concurrence. Rollback criteria were any increase in checkout 5xx, P95 above 2 s for five minutes, or replica health degradation.

Recovery required connected_clients below 80% of effective capacity, blocked_clients returning to baseline, checkout 5xx below 1%, and stable replication for 30 minutes.

Recovery verification was owned by Commerce Platform SRE. The incident record was reviewed on 2026-07-21 and linked to the approval and long-term action tickets so the resolution remains auditable.

Record	Change or action	Owner / approver	Result / due
Approval	CR-REDIS-2026-071, 10% retry-policy canary	Incident Commander + Redis Owner	Approved 10:12 UTC
Action	Cap retry concurrency and add jitter	Order Platform	2026-07-20 / load test
Action	Alert on effective client headroom	SRE Observability	2026-07-24 / alert drill
Action	Set idle pool limits by workload	Runtime Platform	2026-07-31 / config audit
Action	Quarterly capacity review	Redis Service Owner	Recurring / review record